Machine Translated by Google

Тестовое задание для JS-разработчика полного стека (бэкенд).

Создайте API с одним методом GET.

Общие требования: 1. Стек

фреймворков: Nest JS или любой другой JS- фреймворк. TypeScript обязателен.

2. Используйте систему контроля версий и опубликуйте проект на Github.

3. Аутентификация JWT и 3 предопределенных пользователя (администратор, обычный пользователь, пользователь с ограниченными правами).

4. Решение должно генерировать автоматизированную документацию OpenAPI в Swagger или любом другом формате.

другой двигатель.

5. Используйте эту базу данных PostgreSQL в качестве источника данных: https://

rnacentral.org/help/public-database

Данные для подключения к базе данных:

• Имя хоста: hh-pgsql-public.ebi.ac.uk • Порт: 5432

• База данных: pfmegrnargs •

Пользователь: reader

• Пароль: NWDMCE5xdipIjRrp

Описание задачи: 1. Создать

метод GET /locus

2. Параметры фильтрации:

a. Id, int enum (таблица rl) b. assemblyId,

int, single value (таблица rl) c. regionId, enum, (таблица rld) d.

membershipStatus, varchar, single value (таблица

rld)

3. Добавьте параметр боковой загрузки в виде перечисления. В списке должно быть одно значение - locusMembers. 4.

Примените параметры пагинации с количеством строк по умолчанию = 1000.

5. Добавить возможность сортировки результатов для парных полей. 6. Связи

должны быть определены в моделях. Без использования прямого SQL-запроса.

7. Права доступа: администратор имеет доступ ко всем столбцам, обычный пользователь имеет доступ только к данным из таблицы rl и не

может использовать загрузку данных из сторонних источников, ограниченный пользователь может получать

данные только для regionId в (86118093,86696489,88186467)

8. Проведите несколько простых тестов, чтобы проиллюстрировать решение.

Определение таблиц rl и rlm в SQL в виде SQL-запроса: SELECT rl.*,rlm.* FROM

rnc_locus rl LEFT JOIN rnc_locus_members rlm on rlm.locus_id=rl.id

Machine Translated by Google

Пример ответа API без загрузки из сторонних источников (в этих примерах используется SnakeCase, предпочтительнее CamelCase):

[{ "id": 3106326,
"assembly_id": "WEWSeq_v.1.0",
"locus_name":
"cfc38349266a6bc69956bedc917d0edb00069168bf77c8242d50729767e98670@4A/54792566 8-547987324:1",

"public_locus_name": "432B32430F9FCBB8", "chromosome":
"4A", "strand": "1",
"locus_start":
547925668, "locus_stop":
547987324, "member_count":
259, },

{...},
{...},
....
]

Пример ответа API с загрузкой из сторонних источников (в этих примерах используется snake_case, предпочтительнее camelCase):

[

{ "id":3106352,
"assembly_id":"Rrox_v1",

"locus_name":"12b97d958a54d0d01bb023bc95acd722c71a7dd1928a6253af4181bcece31bbe
@KN300177.1/1081562-1081689:-1",
"public_locus_name":"30CA93230012AFC9",
"chromosome":"KN300177.1",
"strand":"-1",
"locus_start":1081562,
"locus_stop":1081689,
"member_count":1,
"urs_taxid":"URS0000A888AB_61622",
"locus_members":[
{
"locus_member_id":3106352,
"region_id":85682522,
"locus_id":2470322,
"membership_status":"member"

Machine Translated by Google

```
        }

  ]},
        {...},
        {...},
        ....
]
```

Дополнительные источники:

Naming / Type convention

> Naming convention is defined differently in each company. The current guidance is defined by global community standards. Some examples of cases may refer to NestJS and NextJS development.

For environment variables each variable, that is defined in Kubernetes and consumed by application should be defined in upper snake-case like **DB_PORT**.

- Do not refer any environment / namespace in the application code. Only if it refers to custom logic of Cron configuration of execution.

For application code in case of defining variables, classnames, functions the name should be in camelCase format like **vehicleId**.

- In case of defining model attributes should be also in camelCase. It may conflict with the table column naming that has snake_case. The TypeORM Strategy of mapping one format into other will help.
- Consider to use full names, don't shorten like `carNr` should be used more as `carNumber`

For database the tables, columns, triggers, unique / foreign key names should be defined in snake_case.

- In case of using UUIDv4 or any other similar format, the column name should refer to **Id** but not **Uuid**.
- Having information regarding date and time should consider having just one column with type **Datetime** emerging both parts.

List of accepted namings of variables, functions, classes etc in our team.

Date range: `startDate / endDate`

Event date: `startedAt / finishedAt` or any verb with `At` - ending e.g: `closedAt / updatedAt`

Requesting data: method name starts with `get`, e.g: `getCars() / getTaskHours()`